



MORROW

Terminal Product

Complete User & Architecture Manual

Private Intelligence, Built Around You

Version 0.1.0-beta.18 — July 03, 2026

Axiom International · Kitchener-Waterloo, Ontario · getaxiom.ca

Aidan Magee — Owner & Lead Supervisor · aidan@getaxiom.ca

Contents

1. Introduction & Why Morrow

Design philosophy, product principles, comparison at a glance.

2. Quick Start

Installation, first launch, one-shot and YOLO modes.

3. The Terminal Shell

Annotated layout diagram with every region explained.

4. Activity & Transparency

Before/after grouping diagram, stage banners, paths.

5. Slash Commands

Complete reference — all 18 commands.

6. Permissions & YOLO Mode

Approval flow diagram, 3 trust levels, hard blocks.

7. Autonomous Workflow & Recovery

Recovery diagram, edit-verify pipeline, 5 options.

8. Mission Control

3-tab cockpit diagram, Ctrl+T navigation.

9. Architecture

Event-driven pipeline diagram, pure reducer pattern.

10. Efficiency & Comparison

Efficiency metrics, tracked statistics.

11. Installation & Build

From-source build, PowerShell quick-install.

12. Consumer Acceptance Test

12-step validation journey.

A. Keyboard Shortcuts

Quick reference card.

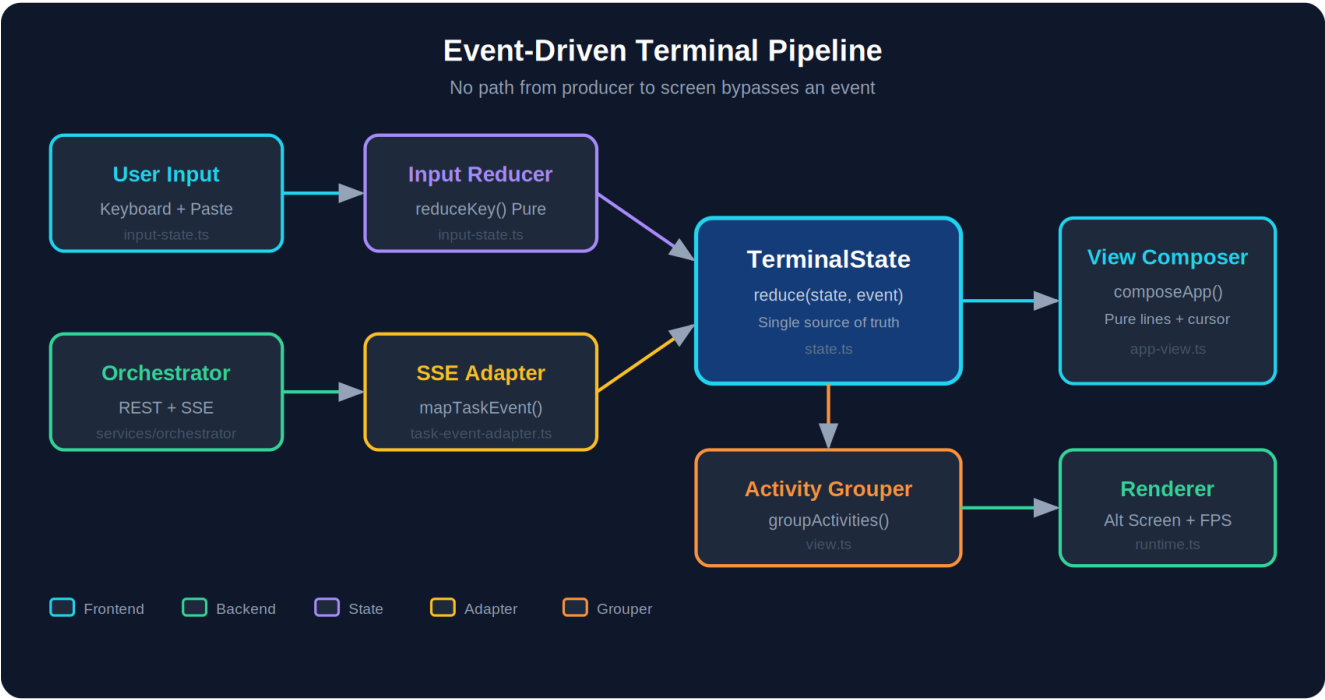
1. Introduction & Why Morrow

Morrow is a self-hosted, deeply customizable personal AI agent built by Axiom International. It lives in your terminal — not a browser tab, not a chat window, not a SaaS dashboard. It runs on your machine, against your files, powered by the model and provider you choose. No vendor lock-in. No telemetry.

Product Principles

- Every unnecessary line is noise. Every raw tool ID is a bug. Every repeated scan is a bug. Every dead command is a bug.
- The active project must always be unmistakable. You always know what Morrow is touching, what mode you are in, and how to recover.
- Routine project work flows without constant interruption. YOLO mode performs normal work automatically while confirming destructive, external, or irreversible actions.

How



Morrow targets the user who wants a finished commercial product. Hermes Agent by Nous Research is an excellent open-source framework — Morrow studies its patterns and builds an original experience that exceeds it in clarity, efficiency, and polish.

2. Quick Start

Morrow runs on Windows 10/11 x64. Install with PowerShell:

```
irm https://getaxiom.ca/morrow/install | iex
```

After installation, open PowerShell 5.1 or Windows Terminal and run:

```
morrow
```

Morrow auto-detects your repository root and Git state. For one-shot queries:

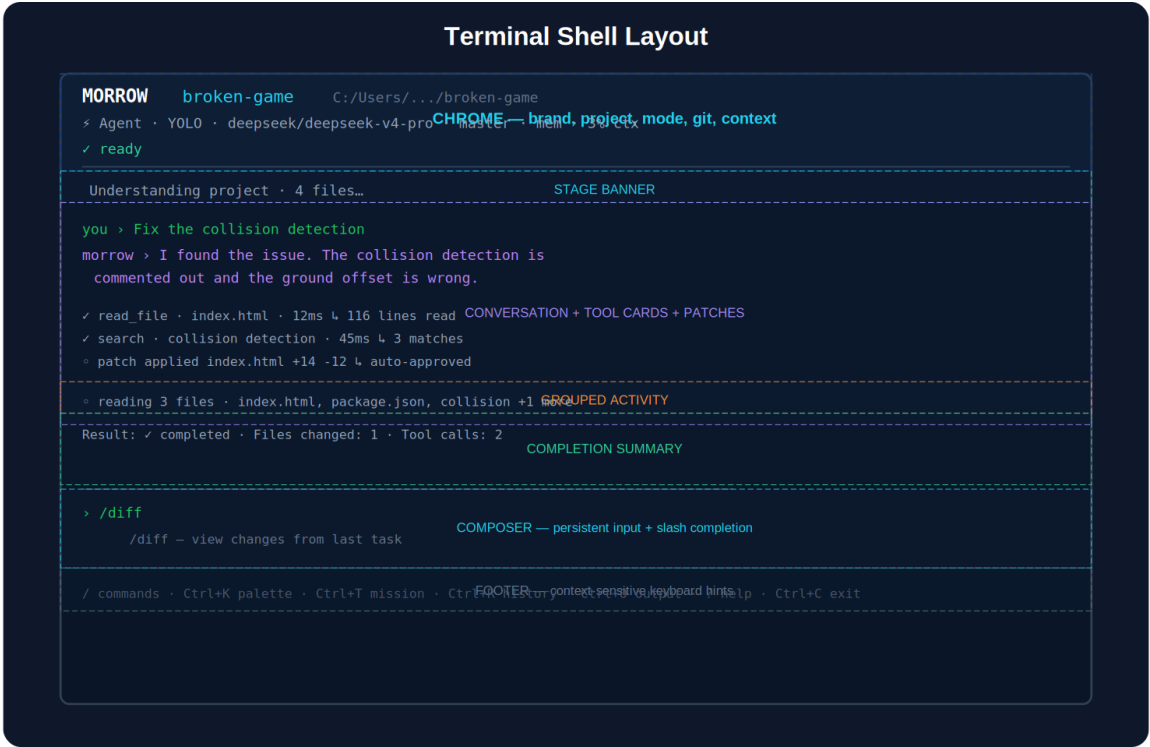
```
morrow ask "Explain this codebase"
```

For autonomous work with auto-approved edits:

```
morrow yolo "Fix all TypeScript errors"
```

3. The Terminal Shell

The default morrow command opens a polished interactive terminal shell. It uses the alternate screen buffer with a capped frame rate. Cursor is hidden during work, restored on exit. Crash recovery is guaranteed — the exit handler always restores the terminal.



The annotated diagram above shows every region. The chrome (top) shows MORROW branding, project in cyan, mode glyph, provider/model, git branch and dirty state, memory toggle, and context percentage. The conversation area renders user messages in green and Morrow responses in magenta. Activity is grouped into compact single-line summaries. The composer has a visual separator, persistent prompt, and slash-command completion. The footer shows context-sensitive keyboard hints.

4. Activity & Transparency

Morrow replaces debug logs with meaningful grouped activity. Successful tool calls are collapsed. Repeated reads and searches are merged. Internal IDs are hidden. Red is reserved for real failures.



Seven raw individual tool-call lines become three compact grouped lines with stage context. Reading + searching + inspecting all merge into the Understanding stage. Each phase gets its own progress banner. Full detail is always accessible through `/output` drill-down.

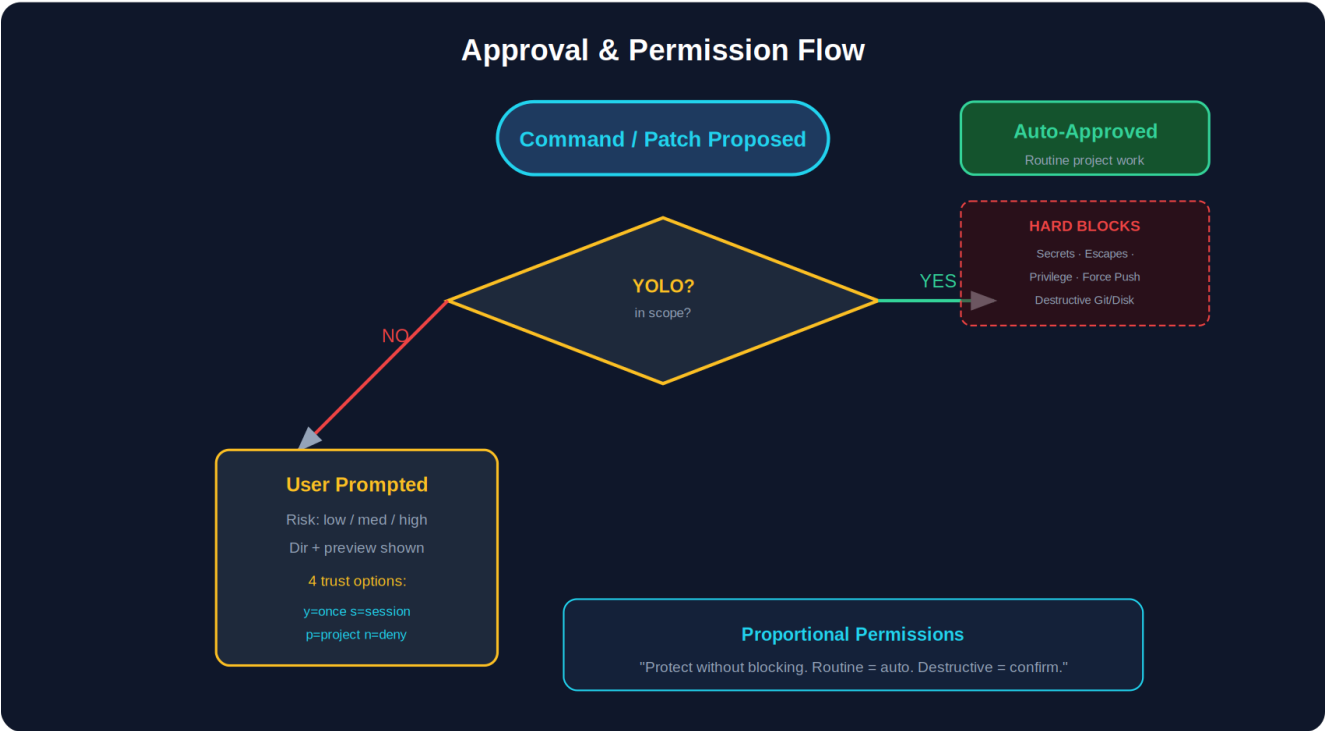
5. Slash Commands

All commands execute natively in the TUI. Zero redirects to line mode.

/help	Show all available commands
/mode [agent inspect plan]	Switch to mode
/yolo [on off status toggle]	Toggle autonomous approval
/model [id]	Show/change active model
/project	Project name, path, branch, mode
/tree, /tasks	Task tree for current mission
/result	Mission aggregate (verification, approvals)
/context	Token usage, compaction, trim
/diff	Diff of last task changes
/undo	Roll back last task file changes
/output [id]	Captured tool output
/panic	Emergency: disable YOLO + cancel task
/status	Current session summary
/memory	Toggle project context memory
/permissions	YOLO state, workspace, mode legend
/resume, /continue	Resume paused task
/exit, /quit	Exit shell

6. Permissions & YOLO Mode

Morrow protects users from accidents without blocking legitimate work. Three trust levels and a YOLO mode for autonomous operation.

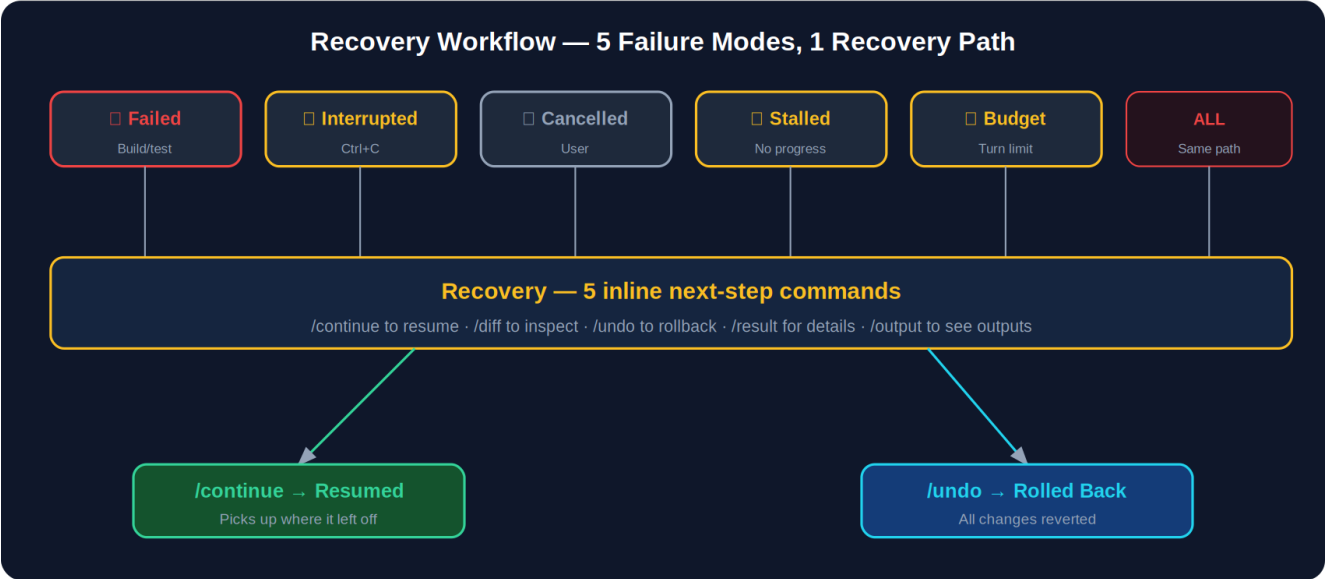


The approval panel shows: the exact operation, working directory, purpose and risk level (color-coded), preview of output or diff changes, YOLO status reminder, and four trust options: approve once, trust session, trust project, or deny.

YOLO mode enables project-scoped autonomy. Hard blocks remain for: secret access, workspace escape, privilege escalation, destructive disk actions, destructive Git, force push, and unauthorized network transmission.

7. Autonomous Workflow & Recovery

Morrow behaves like a strong senior engineer: understand, plan, edit, verify, recover.



Five failure modes (failed, interrupted, cancelled, stalled, budget-reached) all lead to the same Recovery section with five inline commands. Ctrl+C cancels active work. A second Ctrl+C confirms exit. Terminal state always restores correctly.

8. Mission Control

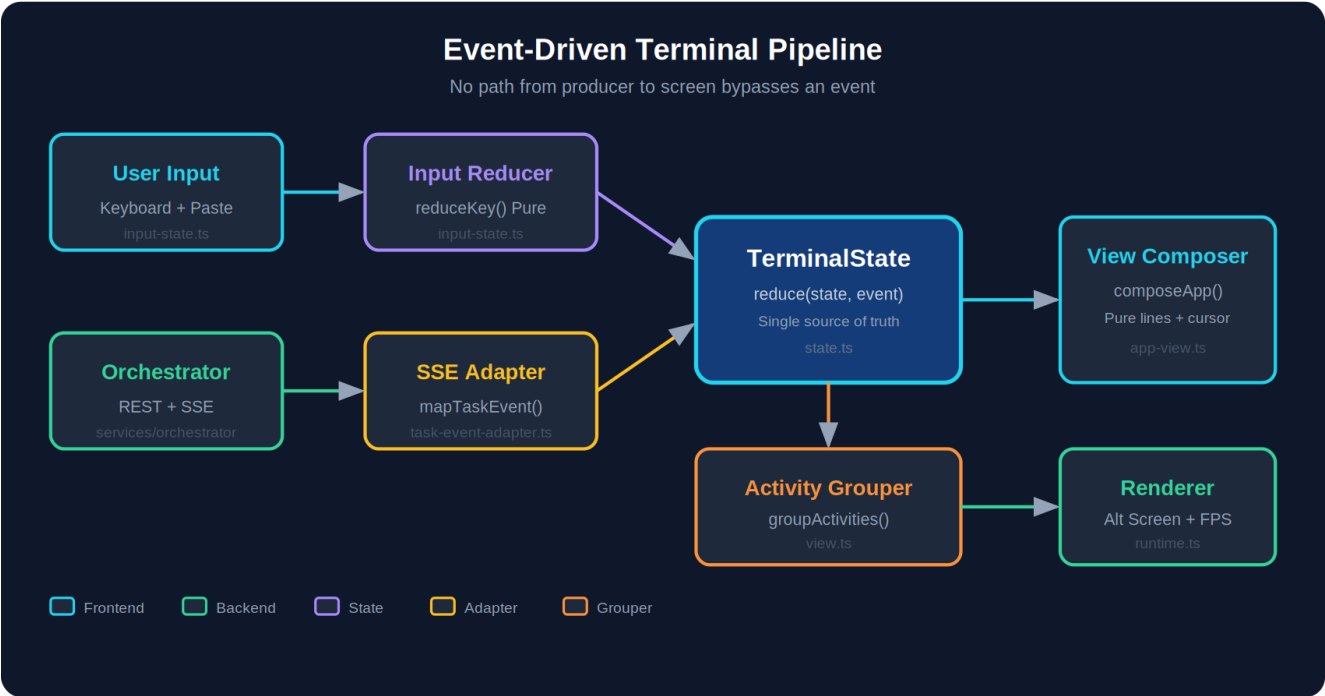
Ctrl+T opens Mission Control — a tabbed operational cockpit with three views loaded in parallel.



Tab 1: Task Tree — hierarchical mission view. Tab 2: Live State — activity, tools, git, context, agents, plan. Tab 3: Mission Result — full aggregate with verification, approvals, integrations. Navigate with arrow keys or 1/2/3. Esc closes.

9. Architecture

Morrow follows a strict event-driven pipeline. Every producer emits a `TerminalEvent`. The reducer folds events into `TerminalState`. The view composer produces pure lines + cursor position. No path from producer to screen bypasses an event.



This decoupling means the entire UI is snapshot-testable: feed events in, assert lines out, no TTY required. Key source files: `session.ts` (event loop), `state.ts` (reducer), `events.ts` (event types), `view.ts` (pure views + grouping), `app-view.ts` (frame composer), `runtime.ts` (renderer), `input-state.ts` (input machine), `task-event-adapter.ts` (SSE bridge), `mission-control.ts` (cockpit), `yolo.ts` (policy).

10. Efficiency

Morrow targets DeepSeek-level efficiency: minimize model calls, tool calls, files inspected, repeated reads, prompt bytes, approvals, and retries. The preferred workflow: reuse cached project summary, use symbol index, read only essential manifests, search narrowly, read relevant files, make targeted edits, verify, summarize.

Morrow tracks: tool-call count, duplicate calls prevented, files inspected, context used, approvals requested, commands run, retries, verification attempts, and time to first useful action. Metrics surface in `/context`, `/result`, and Mission Control.

11. Installation & Build

Requirements

- Windows 10/11 x64 (primary). Linux via source build. macOS planned.
- Node.js >= 22, pnpm >= 9, PowerShell 5.1 or Windows Terminal

```
irm https://getaxiom.ca/morrow/install | iex
```

Build from source

```
git clone https://github.com/Mageester/morrow.git
cd morrow
git checkout product/hermes-parity
pnpm install
pnpm build
```

CLI at bin/morrow.mjs. SQLite at %LOCALAPPDATA%/morrow/state.db.

12. Consumer Acceptance Test

Validate Morrow with this 12-step journey from a fresh PowerShell 5.1 window:

1. Run `morrow`. Complete onboarding if first launch.
2. Select the broken-game project (fixtures/broken-game).
3. Ask: "Explain this project and identify the bugs."
4. Ask: "Repair the collision detection and ground offset bugs."
5. Review activity — observe grouped stages, no raw IDs.
6. Review approval prompts — note risk indicators, trust options.
7. Use `/context`, `/tree`, `/diff`, `/output`, `/result`, and `/undo`.
8. Open Mission Control with `Ctrl+T`. Navigate all three tabs.
9. Resume with `/continue` if interrupted.
10. Run the repaired project in a browser.
11. Exit with `Ctrl+C` twice. Verify terminal restoration.
12. Reopen PowerShell. Run `morrow resume`. Verify persistence.

Appendix A: Keyboard Shortcuts

Ctrl+C	Cancel work (1st) / Exit (2nd, idle) / Close modals
Ctrl+T	Open Mission Control (3-tab cockpit)
Ctrl+K	Command palette
Ctrl+L	Full-screen repaint
Ctrl+R	Search command history
Ctrl+O	Output viewer
?	Help on empty buffer
Up/Down	Recall history
Tab	Complete slash command
Esc	Close overlay
Left/Right / 1-3	Navigate Mission Control tabs
y/s/p/n	Approval: once / session / project / deny

Built by Axiom International

Private Intelligence, Built Around You

Kitchener-Waterloo, Ontario · getaxiom.ca

aidan@getaxiom.ca · 226-753-1833

